Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/1937728.1937775

PDF Download
1937728.1937775.pdf
29 March 2026
Total Citations: 2
Total Downloads: 181

Published: 30 December 2010

Citation in BibTeX format

ICIMCS '10: The Second International Conference on Internet Multimedia Computing and Service
December 30 - 31, 2010
Harbin, China

RESEARCH-ARTICLE

Algorithm for generating snow based on GPU

CHENGMING ZOU, Wuhan University of Technology, Wuhan, Hubei, China

XIUFENG XIE, Wuhan University of Technology, Wuhan, Hubei, China

GUANGHUI ZHAO, Wuhan University of Technology, Wuhan, Hubei, China

Open Access Support provided by:

Wuhan University of Technology

ICIMCS '10: Proceedings of the Second International Conference on Internet Multimedia Computing and Service (December 2010)
https://doi.org/10.1145/1937728.1937775
ISBN: 9781450304603

Algorithm for Generating Snow based on GPU

Chengming Zou
Wuhan University of Technology

Xiufeng Xie
Wuhan University of Technology

Guanghui Zhao
Wuhan University of Technology

xxf224@126.com

ABSTRACT
The simulation of natural phenomena plays an important role in the area of computer game, film production, simulation and so on. Snow is an important part of the simulation of nature phenomena, so how to generate high-quality and controllable snow texture is significant. Now the algorithms of generating snow are based on triangulated surfaces, they put some triangles on multi-layer concentric spheres to form snow. The disadvantage of these algorithms is that the edges of snow are not smooth enough, the angles of snow are too acute. The approach is splitting up a triangle to four, and then rendering the triangles associated with vertexes using the rendering algorithm of Quadric Bezier Curve based on GPU, and rendering the triangle formed by points on edges normally. Finally it can generate high-quality snow texture with the technology of alpha blending.

Keywords
Snow texture, Bezier curve, GPU rendering, real time.

1. INTRODUCTION
In computer graphics, simulation of natural phenomena has always been popular and challenging. As the study results can be widely used in computer games, video production, computer simulation and other fields, so realistically simulating snow is very valuable.

Now it mainly has the following four aspects for snow simulating: the process of snowfall, snow accumulation, snow stability and snow mark model. Robert Sumner and others [3] modeled a deformable material by a high degree of domain, and then used this material to determine the height of the mesh vertices to achieve snow mark effect. Paul Fearing [2] made simulation mainly from the two aspects of snow accumulation and snow stability, proposed the accumulation model and stability model. T.B. Moeslund and others [4] proposed a general snowfall and snow accumulation model appearance, trajectory and accumulation shape of snow are all established in the physical model of real world, so it is controlled by various weather parameters, such as: temperature, humidity and wind direction, etc. It is very attractive to model the behavior of snowflakes on the physical model, because it allows us to dynamically change the various properties of snowflake, and enable users dynamically interact with games and simulation software. The 3-D snowflakes generation algorithm of T.B. Moeslund [4] described the appearance of snowflakes vividly, and its effect is better than the simulated snowflakes of any existing film scenes. However, as the high real-time requirements of game software, it appears unrealistic of rendering 3-D snowflake. So we transplant the algorithm to 2-D space, dynamically generating snowflakes texture pictures, and then treat them as textures of a particle system, rendering the process of a snowfall. Watching the process of a snowfall from far away, for example, in the house, the effect is very realistic. But if we walk into the snow, watch the snowflakes closely, we will find that there will be sharp corners at the edges of snowflakes, and the cohesion of the edges of snowflakes aren't smooth enough. Considering the disadvantages, the paper proposed a new method: smoothing the edges of the snowflake by making use of the algorithm of Bezier curve based on GPU rendering. As the generation of snowflakes vertex coordinates gets help from the snowflakes generation algorithm of T.B. Moeslund [4], the appearance and characteristics of snowflakes can be controlled by weather parameters.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.
ICIMCS'10, December 30-31, 2010, Harbin, China.
Copyright 2010 ACM 978-1-4503-0460-3/10/12…$10.00

2. THE SNOWFLAKE GENERATION ALGORITHM BASED ON TRIANGULAR POLYGON
The snowflakes generation is very complex, and is hard to understand thoroughly. Snowflakes are formed by ice crystals, and the generation of ice crystals must meets the following five conditions at the same time: moisture, condensation nuclei, water in the clouds, temperature and condensation shape. How the ice crystals form snowflakes depends on two factors: temperature and humidity.

The three factors which affect the appearances and characteristics of the snowflakes are: shape, size and density. So we can establish relationship between the three factors and temperature or humidity. The article of Charles Loop [5] established the functional relationship between the radius of the snowflake and temperature, as the following equation:

$$D = \begin{cases} 0.015 \cdot |T|^{-0.35} & (T \le -0.061) \\ 0.04 & (T > -0.061) \end{cases} \quad (1)$$

In the equation: D represents the diameter of a snowflake, and T represents the Celsius temperature. The equation (1) shows the relationship between the temperature and the average diameter, and when snowflakes are generated, we can randomly change the diameter of the snowflake by 50% to produce snowflakes of various diameters. And in the real world, the diameters of the snow and appearance are almost different.

The study shows that there is inverse relationship between the density and diameter of the snowflakes. For dry and wet snowflakes, the ratio of the density and the diameter of the snowflakes is a constant, as following equation (2):

$$\rho_{snowflake} = \frac{C_{humidity}}{D} \quad \textbf{(2)}$$

$$C_{dry} = 0.170 \frac{kg}{m^2} \quad \textbf{(2a)}$$

$$C_{wet} = 0.724 \frac{kg}{m^2} \quad \textbf{(2b)}$$

In the equation, $\rho_{snowflake}$ indicates the density of the snowflakes, and C is constant, in dry conditions 0.170 kg/m^2, and in wet conditions 0.724 kg/m^2. D represents the diameter of a snowflake.

In the real world, snowflakes are formed by collision of ice crystals. Inspired by this, T. B. Moeslund and others [4] presented a method of randomly combinating of triangular polygon to form snowflake. The article used a series of concentric spheres, and each concentric sphere as a layer, placing the same number of triangles in each layer, as show in Figure 1. The number of concentric sphere layers controls the diameter of the snowflakes, and placing the same number of triangles on each layer to control the density implicitly.

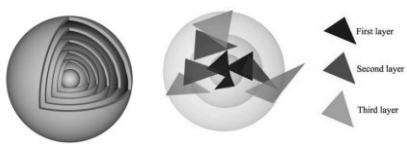
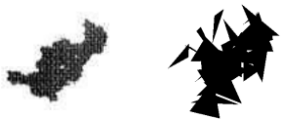


Figure 1. Add triangles to different layers of concentric.



(a) (b)

Figure 2. Left is real snow, right is synthetic snow.

By comparing the left picture and the right picture of Figure 2, the snowflake which is generated by the program is very similar to the real snowflake, which is very satisfactory. But we can find by observation that the corner angles of snowflake which are generated by the program are sharper than real ones. Especially for wet snowflakes whose radiuses are smaller, the problem is especially serious.

T.B. Moeslund and others [4] had paid attention to this problem, so he suggested using some methods of rendering to decrease the advantages. Although the article didn't describe the method of rendering, we can declare that it is Alpha blending from the snowflake pictures provided by the article, as show in Figure 3.

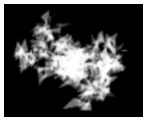


Figure 3. Snow with alpha blending.

Figure 4. Dry snow.

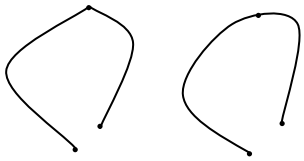
However, the performances are not satisfactory, especially for some dry and small snowflakes which still have many sharp corner angles, as show in Figure 4.

3. SMOOTH THE EDGE OF SNOWFLAKE

The general process of algorithms of this paper can be divided into three parts. First, generating vertices coordinates of the triangles which form the snowflakes with the algorithm for generating snowflakes based on triangular polygons, and then dividing or expanding each triangle. At last, render the snowflakes with the Bezier Curve rendering algorithm based on GPU.

Aagaard and others [6] had given the algorithm for generating snowflakes based on triangular polygons. By this algorithm, we can get the vertex coordinates of the triangles which form the snowflakes.

We have acquired the linked lists of the vertex coordinates of the triangle which forms snowflakes, and the object of operation of the Bezier curve rendering algorithm based on GPU is single triangular polygon. So we need to smooth each triangle. Every triangle has three corners, so we need three Bezier curves to smooth each corner. Therefore, the triangle needs to be divided into some small triangles. When the triangles are divided or expanded, the following condition must be met: the join of two Quadratic Bezier curves must be smooth and jumping point can not appear, as shown in Figure.5. Otherwise, it is possible that there will be sharp corners on the edges of the snowflakes.



(a) **(b)**

Figure 4. Left is the join of 2 curves with jump point, Right does not have jump point.

The condition of whether the convergence of two Bezier curves is smooth is: the tangent vectors of points which connect two Bezier curves must be parallel. Any splitting points we get from the sides of triangles can meet the condition, because the splitting points are on the side formed with two vertices of the triangle, the three points collinear. We will provide the process of the triangle splitting algorithm as following:

1. Initialize a linked list newList. The list is used to store the space coordinates and texture coordinates of triangles after decomposition;

2. Loop through the linked list oldList that is generated by the algorithm of snowflakes generation based on triangle, getting three vertex coordinates A0、A1、A2 each time;

3. Get three points randomly from the three sides: M0、M1、M2;

4.Add the triangle formed by the vertex A0 and points M1, M2 on the adjacent sides into the linked list newList. Set the texture coordinate of vertex as (0.5, 0.0), and texture coordinates of points M1 and M2 as (0.0, 0.0) and (1.0, 1.0);

5. Repeat the step 4, and add the triangle formed by the points A1, A2 and points on the adjacent sides into the linked list newList;

6. Add the triangle formed by the points M0, M1, M2 into the new linked list. The value of the texture coordinates of the triangle M0M1M2 must meet the condition U2 – V ≤ 0, and we generally set it as (0.0, 0.0);

7. If the linked list oldList is not empty, return to the step 2; if the linked list is empty, jump to the step 8;

8. Observe the effects of the snowflakes generated. If the effect is not perfect, then return to the step 1, and choose different points M0, M1, M2 again.

The effect of the algorithm above shows as Figure.6:

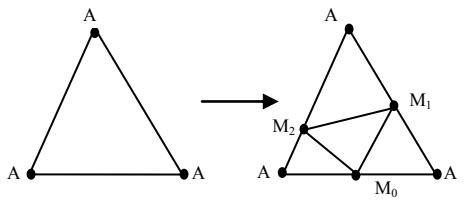


Figure 5 Triangle split up.

There is a disadvantage with this algorithm for triangle splitting: the area of each triangle which had been smoothed will be smaller than the old triangle, and thus there will be many spaces between snowflakes generated. And there is another proposal. Expand each triangle into four triangles, render new generated triangles with Bezier curve rendering algorithm, and render the former triangles conventionally. It is a special form as shown in Figure.7. In the figure, each expansion point is the symmetric point of one triangle vertex to the opposite side, but the area of the triangle smoothed by Bezier curves will be larger than former triangles, generally about 3 times.

In order to decrease the difference between the area of the triangle expanded and the former triangle, we put forward two proposals. The first proposal: decrease coordinate of each vertex of the triangle, decreasing the area of the former triangle to one third of that, and then expand the triangle by the symmetric points. As a result, it lost flexibility when new triangles are generated, and there will be the same disadvantage as the splitting algorithm: many spaces between snowflakes generated. Of course we can remove the disadvantage by adjusting the decreasing ration of the vertex coordinates properly, but the choice of decreasing factor is very hard to master.

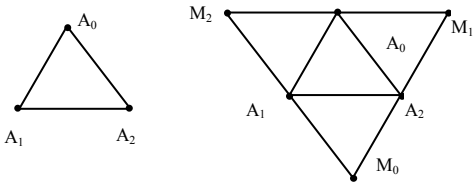


Figure 6. Triangle expand.

The second proposal is translating the expansion points to the vertex of the triangle. For example, translate the point M1 in Figure 7 to the point A0, and the point M2 to the point A1. But the disadvantage is the two expansion points and the vertexes of the triangle are not in the same horizontal line. For example, M1, A0 and M2 are no longer in the same line, so the joining of the Bezier curves is not smooth anymore, generating jumping points. But we found in the practical experiments, the negative effect that the convergence of Bezier curves is not smooth is very weak. The proposal brings much flexibility: we can adjust the control points dynamically to find the best expansion point. The process of the algorithm for the triangle expansion is similar to the triangle splitting algorithm.

We can get a linked list including vertex coordinates and texture coordinates. In the next step, we need to compile an algorithm based on GPU. We have known how to render the Bezier curves of a triangle by GPU, and next we only need to apply the algorithm here. Because the triangles in the linked list are placed by the following order: A0M1M2, A1M2M0, A2M1M0, M0M1M2, we first need to render the Bezier curves of the first three triangles, and then render the last triangle conventionally. When rendering we should shut down culling .

But the GPU algorithm can be optimized further: First measure whether the triangle is formed by vertices and split points (expansion points) or all formed by split points (vertices). We have already put the judgment condition into the texture coordinates, and make the texture coordinates of the triangle formed by split points or vertices meet the condition: U2 – V <= 0.0, such as (0.0, 0.0), and the algorithm complexity can be simplified greatly. The simplified algorithm is as following:

IF (the texture coordinates following pixels of the triangle meet the condition: U2 – V <= 0.0):

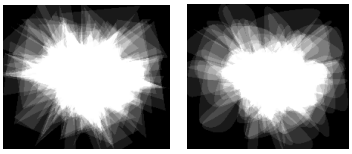
Set the Alpha value of the pixel as 1

ELSE：

Set the Alpha value of the pixel as 0

4. THE RESULT OF THE EXPERIMENT AND ANALYSIS

First, we generate the triangles, which form snowflakes, by the algorithm of T.B. Moeslund [4] and then render them. The effect is shown as Figure(a) of Figure.8. Then we split the triangles, and smooth the edges of the snowflakes. The effect is shown as Figure(b) of Figure.8. We can see the edges of the snowflake are almost smoothed completely and no sharp corner turn up.



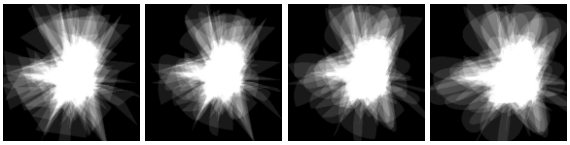
(a) **(b)**

Figure 7. Left is unsmoothed snow, Right smoothed.

In order to generate the best snowflake texture, we choose split points by different parameters for the algorithm. The snowflake which is not smoothed is shown as Figure(a) of Figure.9; Figure(b) of Figure.9: the first and the third split point coincide with the vertices, and the second split point is on the one fifth of the corresponding side; Figure(c) of Figure 9: the first split point is on the one tenth of the corresponding side, and the second and third split point are two fifth of the corresponding sides; Figure(d) of Figure 9: the three split points are on the one half of the corresponding sides, known as middle point splitting. By

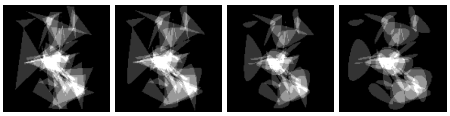
comparison we can find that the split point of Figure(c) is better, because the edges of the snowflakes are smooth, and the sharp corners have been passivity. Although the sharp corners also have been passivity in Figure(d), the edges have not been smoothed perfectly.

The effect of smoothing big snowflakes by split algorithm is very obvious, but not very perfectly for small snowflakes. Snowflakes are smoothed by the same split parameters of Figure 9, and the effect is shown as Figure.10. We find the effect of these split parameters is not very good.



(a) (b) (c) (d)

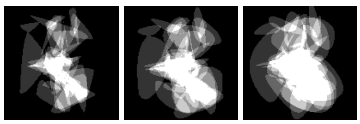
Figure 8. Smooth big snow with different splitting factor.



(a) (b) (c) (d)

Figure 9. Smooth small snow with different splitting factor.

If the expansion algorithm is used, the effect will be better, shown as Figure.11. In Figure(a) of Figure 11, the expansion coefficient is 0.2; In Figure(b) of Figure 11, the expansion coefficient is 0.5; In Figure(c) of Figure 11, the expansion coefficient is 1.0.

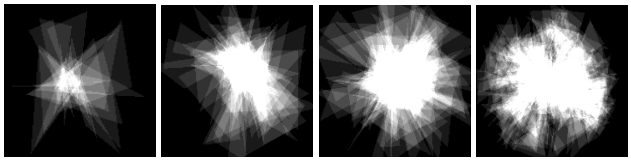


(a) (b) (c)

Figure 10. Smooth Snow with different expansion factor.

The shape of Figure(a) is close to that of the original figure, but the effect of smoothing is not very obvious; The shape of Figure(c) is the smoothest, but its area is larger than that of the original snowflake. We can choose the snowflake of Figure(b) in the middle way.

In order to obtain vivid snowflakes, we can place different number of triangular patches on each layer of concentric spheres to obtain the best effect. We only need to guarantee that the number of triangles of each layer when dry is one fourth of that when wet. From Figure(a) to Figure(d) in Figure.12, the number of triangles placed on each layer is separately: 10, 20, 40 and 70. By observation we can find that the effect is better if 20 triangles are placed on each layer when dry.



(a) (b) (c) (d)

Figure 11. Put different triangles on per layer.

The snowflakes which are generated by our algorithm are used as the textures of particle system, and the process of rendering snowfall is shown as Figure 13. Because the Alpha blending is taken, we find the edges of the snowflakes are vague. So the snowflakes seem not real. But the process of snowflakes landing is continuous; the effect is very perfect when the continuous process of snowflakes landing is observed.

5. CONCLUSION

The paper proposals an algorithm for generating snowflake texture based on GPU. The characteristics of the algorithm are: making the edges of the snowflakes more smooth, and the shape and attribute of snowflakes are controlled by weather parameters, such as temperature, humidity and so on. The algorithm of this paper can be improved properly following needs: for example, the anti-aliasing technique can be included. Because the snowflake texture picture is not very large, we don't add anti-aliasing technique in order to improve efficiency.(For more details, please read the book of Hubert Nguyen [7]). In our algorithm, the triangles which form snowflakes are generated by CPU first, and then these triangles are rendered by GPU. When there are many triangles, parts of the triangles can be calculated first by CPU, and then rendered by GPU. The process is repeated as so, making the CPU and GPU working parallel.

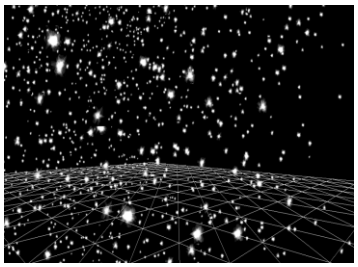


Figure 12. Snow Falling process.

6. REFERENCES

[1] Tomas Akenine-Moller, Eric Haines, Naty Hoffman. Real-Time Rendering[M]. 3th Edition. Wellesley: A K Petrs Ltd, 2008: 578-584;

[2] Paul Fearing. Computer Modelling Of Fallen Snow. ACM SIGGRAPH 2000 Conference Proceedings. 2000,37-46;

[3] Robert W. Sumner, James F. O'Brien, Jessica K. Hodgins. Animating Sand, Mud, and Snow. In Computer Graphics Forum. 1999. 18(1):17-2;

[4] T.B. Moeslund, C.B. Madsen, M. Aagaard, and D. Lerche. Modeling Falling and Accumulating Snow[C/OL].(2005) [2010-5-2].http://www.cvmt.dk/Snow/snow.pdf;

[5] Charles Loop, Jim Blinn. Resolution Independent Curve Rendering using Programmable Graphics Hardware. Proceedings of ACM SIGGRAPH 2005. 2005, 24(3): 1000-1009;

[6] Aagaard, Michael, Lerche, Dennis. Realistic Modeling of Falling and Accumulating Snow[D].Denmark: Aalborg University, 2004;

[7] Hubert Nguyen. CPU Gems 3[M]. Boston: Pearson Education Inc, 2008: 543-56.